Centro Universitário - IESB

Engenharia de Software

DAVID JEIEL DE ARAÚJO ROCHA

OTIMIZAÇÃO DE ROTAS LOGÍSTICAS COM TEORIA DOS GRAFOS

solução em Python para a empresa de transporte de Carlos

Brasília

2026

David Jeiel de Araújo Rocha

OTIMIZAÇÃO DE ROTAS LOGÍSTICAS COM TEORIA DOS GRAFOS

solução em Python para a empresa de transporte de Carlos

Relatório da Atividade Ativa apresentado à disciplina de Teoria dos Grafos e Análise de Algoritmos do Curso de Graduação Engenharia de Software – Centro Universitário IESB, como requisito parcial de avaliação.

Orientador: Prof. Francisco Filho.

Brasília

2026

SUMÁRIO

1 INTRODUÇÃO 4

2 ARQUITETURA PROPOSTA 5

2.1 Componentes e bibliotecas utilizados 5

2.2 Modelagem do problema 5

2.3 Funções do software 6

2.4 Algoritmos empregados 6

2.5 Execução 7

3 RESULTADOS 8

3.1 Grafo dos pontos de entrega 8

3.2 Menor caminho entre o Lago Norte e o Lago Sul 8

3.3 Custo dos caminhos entre a sede e os pontos de distribuição 9

3.3.1 Menor custo da sede até cada ponto 10

3.3.2 Todos os caminhos possíveis 10

3.3.3 Árvore Geradora Mínima 11

3.4 Rota mais econômica partindo do Lago Norte 11

3.5 Custo em reais: da Esplanada, por todos os pontos, até o Lago Norte 12

4 CONCLUSÃO 14

REFERÊNCIAS 15

1 INTRODUÇÃO

A Teoria dos Grafos é o ramo da matemática discreta que estuda relações entre os elementos de um conjunto. Um grafo é definido como um par $G = (V, E)$, em que V é um conjunto finito e não vazio de **vértices** (ou nós) e E é um conjunto de **arestas**, pares de vértices que indicam a existência de uma ligação entre eles (GOLDBARG; GOLDBARG, 2012). Essa abstração simples permite representar qualquer situação em que objetos se conectam: cidades ligadas por estradas, computadores ligados por cabos ou pessoas ligadas por amizade.

Entre as principais características usadas para classificar um grafo, destacam-se:

- **Direção:** em um grafo *não direcionado* a aresta $\{u, v\}$ pode ser percorrida nos dois sentidos; em um *dígrafo* cada arco possui sentido único.
- **Ponderação:** em um grafo *ponderado* cada aresta recebe um peso por meio de uma função $w: E \rightarrow \mathbf{R}$, que pode representar distância, tempo ou custo.
- **Grau:** o grau $d(v)$ de um vértice é o número de arestas que incidem nele.
- **Conexidade:** um grafo é *conexo* quando existe um caminho entre qualquer par de vértices.
- **Caminhos e ciclos:** um *caminho* é uma sequência de vértices ligados por arestas; quando não repete vértices é dito *simples*, e quando passa uma única vez por todos os vértices é chamado de *hamiltoniano*.

As aplicações são numerosas: sistemas de navegação e roteamento de veículos usam algoritmos de caminho mínimo como o de Dijkstra; protocolos de redes (como o OSPF) calculam rotas de pacotes sobre grafos; redes sociais, dependências entre tarefas de um projeto e malhas elétricas também são modeladas dessa forma (CORMEN *et al.*, 2012). Na logística, um problema clássico é o do **Caixeiro Viajante**, que busca a ordem de visita de menor custo a um conjunto de pontos.

Este relatório apresenta o software desenvolvido pela VOCÊ SA para a empresa de transporte de Carlos que, diante da alta dos combustíveis, precisa escolher os percursos mais econômicos entre seus seis pontos de entrega em Brasília. A rede foi modelada como um grafo não direcionado e ponderado, e o programa, escrito em Python, responde às cinco questões propostas na atividade, documentadas na Seção 3.

2 ARQUITETURA PROPOSTA

O software foi escrito em Python 3 em um único módulo, `solucao_transporte_carlos.py`, organizado em funções independentes que separam três responsabilidades: modelagem do grafo, algoritmos de otimização e visualização. Essa separação permite testar cada parte isoladamente e reaproveitar as funções: todos os números apresentados neste relatório vêm diretamente da execução do código.

2.1 Componentes e bibliotecas utilizados

Tabela 1 – Bibliotecas, módulos e funções de terceiros empregados

Componente	Tipo	Uso no software
NetworkX 3.x	Biblioteca de terceiros	Estrutura do grafo (*nx.Graph*), Dijkstra (`single_source_dijkstra`, `all_pairs_dijkstra`), enumeração de caminhos (`all_simple_paths`), Árvore Geradora Mínima (`minimum_spanning_tree`, algoritmo de Prim) e funções de desenho.
Matplotlib	Biblioteca de terceiros	Renderização das figuras e exportação em PNG (*pyplot.subplots*, *savefig*).
itertools	Módulo nativo	*permutations*: gera todas as ordens de visita para o problema do Caixeiro Viajante.
os / sys	Módulos nativos	Criação da pasta de saída e gravação do log do console como evidência dos testes.

Fonte: elaborado pelo autor.

2.2 Modelagem do problema

A malha de entregas foi modelada como o grafo $G = (V, E, w)$, não direcionado, conexo e ponderado, com $|V| = 6$ vértices e $|E| = 10$ arestas. Cada vértice é um ponto de entrega e o peso de cada aresta é o custo informado na Figura 1 do enunciado, que agrega distância, velocidade da via e congestionamento. No código, as arestas ficam em uma lista de tuplas (origem, destino, peso), inserida de uma só vez no grafo:

```
ARESTAS = [("Asa Sul", "Lago Sul", 5), ("Asa Sul", "Esplanada", 5),
           ("Asa Sul", "Asa Norte", 12), ("Lago Sul", "Esplanada", 11),
           ("Lago Sul", "Vila Planalto", 14), ("Esplanada", "Asa Norte", 6),
           ("Esplanada", "Vila Planalto", 10), ("Esplanada", "Lago Norte", 8),
           ("Asa Norte", "Lago Norte", 12), ("Vila Planalto", "Lago Norte", 7)]

def construir_grafo():
    G = nx.Graph()
    G.add_weighted_edges_from(ARESTAS)
    return G
```

Internamente, o NetworkX armazena o grafo como uma lista de adjacência baseada em dicionários, o que dá acesso em tempo constante ao peso de qualquer aresta

(`G[u][v]["weight"]`). Para que o desenho reproduza a disposição da Figura 1, as coordenadas de cada vértice foram fixadas no dicionário `POSICOES`, em vez de usar um layout automático.

2.3 Funções do software

Tabela 2 – Funções que compõem o software e sua responsabilidade

Função	Responsabilidade
`construir_grafo()`	Cria o grafo ponderado a partir da lista de arestas.
`plotar_grafo()`	Desenha vértices, arestas e pesos; destaca em vermelho uma rota, numerando a ordem dos trechos, e marca a origem (verde) e o destino (vermelho).
`menor_caminho()`	Item 3.2: aplica Dijkstra entre dois vértices e retorna rota e custo.
`custos_a_partir_da_sede()`	Item 3.3: menores custos da sede a cada ponto, todos os caminhos simples e a Árvore Geradora Mínima.
`otimizar_rota_completa()`	Itens 3.4 e 3.5: busca exaustiva sobre as permutações para achar a ordem de visita de menor custo (Caixeiro Viajante).
`custo_rota()`, `brl()`, `formatar_rota()`	Utilitários: soma dos pesos de uma rota, formatação em reais e exibição da rota com os pesos.
`main()`	Orquestra a execução dos cinco itens, imprime os resultados e salva as figuras.

Fonte: elaborado pelo autor.

2.4 Algoritmos empregados

Dijkstra (itens 3.2 e 3.3). Para caminhos mínimos entre pontos, usou-se o algoritmo de Dijkstra, que parte da origem e, a cada passo, fixa o vértice ainda não visitado de menor distância acumulada, atualizando a distância dos seus vizinhos. Com fila de prioridade, sua complexidade é $O((|V| + |E|) \log |V|)$, e ele é correto porque todos os pesos do problema são positivos (CORMEN *et al.*, 2012).

Busca exaustiva (itens 3.4 e 3.5). Visitar todos os pontos com o menor custo é uma variação do Problema do Caixeiro Viajante, que é NP-difícil: não se conhece algoritmo polinomial para ele. Como a rede tem apenas seis vértices e a origem é fixa, há no máximo 5! = 120 ordens de visita, que o programa avalia integralmente com `itertools.permutations`, garantindo o ótimo global em menos de um milissegundo. Cada ordem é avaliada de duas formas:

- **Caminho hamiltoniano estrito:** só vale se todos os pontos consecutivos forem ligados por aresta direta, e nenhum ponto se repete;

- **Passeio ótimo:** entre duas paradas o veículo usa o menor caminho de Dijkstra, podendo atravessar de novo um ponto já visitado quando isso for mais barato — situação comum na prática, já que passar por um bairro não exige nova entrega.

```
for perm in itertools.permutations(intermediarios):
    seq = [origem, *perm] + ([destino_final] if destino_final else [])
    pares = list(zip(seq[:-1], seq[1:]))
    if all(G.has_edge(u, v) for u, v in pares):      # caminho hamiltoniano
        c = custo_rota(G, seq)
        if c < melhor_estrita[1]: melhor_estrita = (seq, c)
    c = sum(dist[u][v] for u, v in pares)            # passeio (Dijkstra)
    if c < melhor_passeio[1]: ...
```

Prim (item 3.3). Como visão complementar do custo da rede a partir da sede, calculou-se a Árvore Geradora Mínima, o menor conjunto de ligações que mantém todos os pontos conectados sem ciclos.

2.5 Execução

Após instalar as dependências (`pip install networkx matplotlib`), basta executar `python solucao_transporte_carlos.py`. O programa imprime os resultados de cada item no console, grava uma cópia desse texto em `saidas/saida_console.txt` e salva as cinco figuras apresentadas a seguir na mesma pasta.

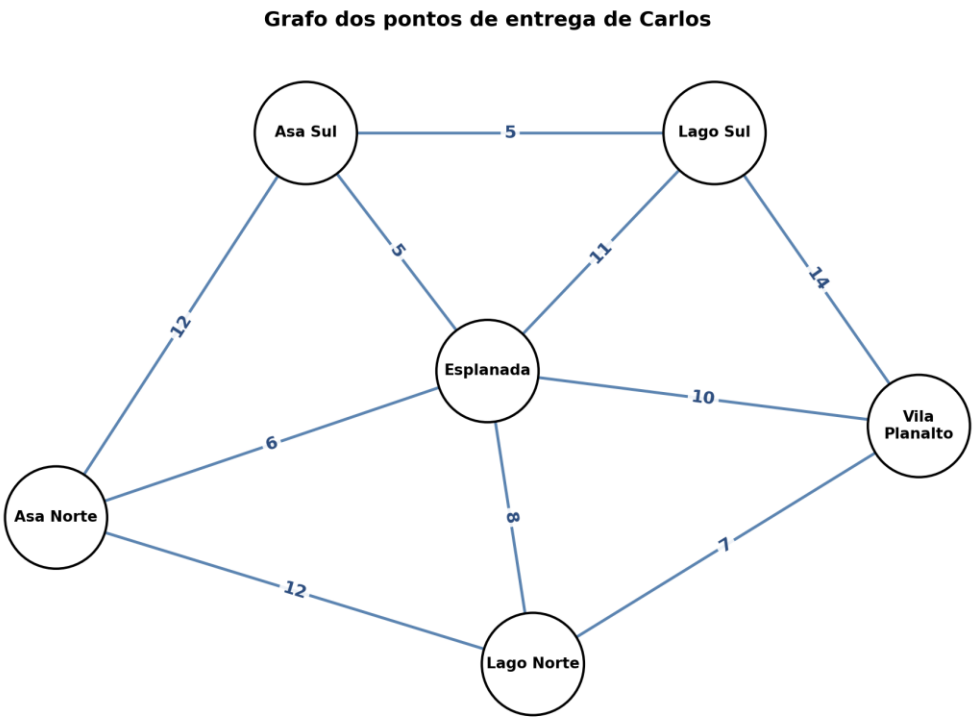
3 RESULTADOS

Esta seção apresenta a saída do programa para cada item solicitado no enunciado.

3.1 Grafo dos pontos de entrega

A Figura 1 mostra o grafo montado em Python, com seus 6 vértices, 10 arestas e respectivos pesos. A Esplanada funciona como ponto central da rede: tem grau 5, ligando-se diretamente a todos os demais pontos, enquanto cada um dos outros vértices tem grau 3.

Figura 1 – Grafo dos pontos de entrega de Carlos gerado em Python



Fonte: elaborado pelo autor (saída do programa em Python).

3.2 Menor caminho entre o Lago Norte e o Lago Sul

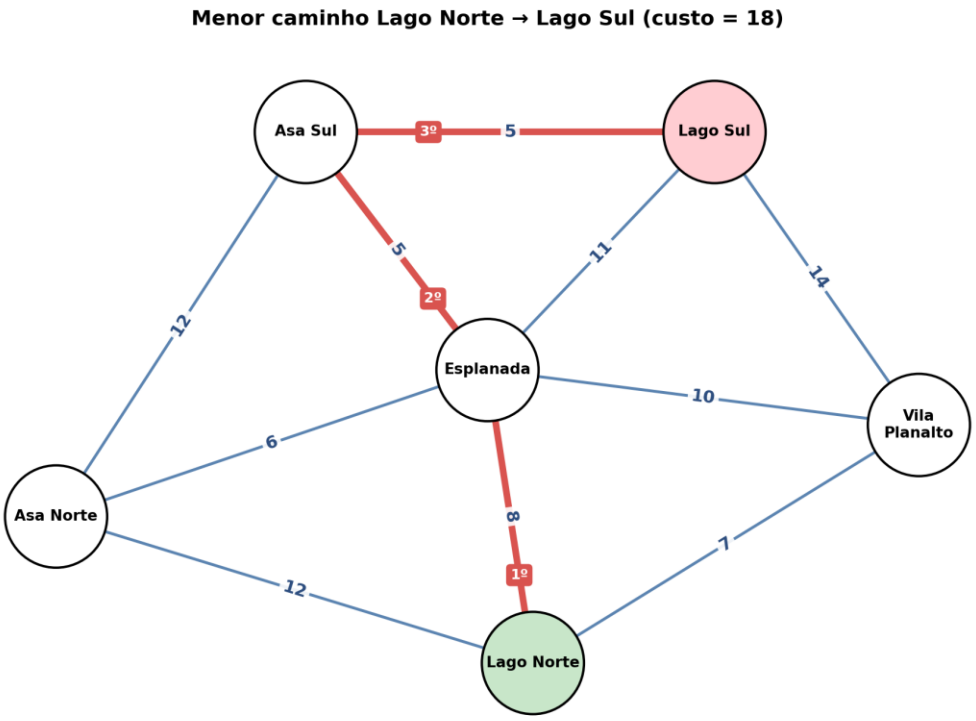
Aplicando o algoritmo de Dijkstra com origem no Lago Norte, o programa encontrou como rota de menor custo **Lago Norte → Esplanada → Asa Sul → Lago Sul**, com custo total **8 + 5 + 5 = 18**. Chama atenção que o caminho ótimo não é o que usa menos arestas: ir pela Esplanada e seguir direto ao Lago Sul custaria 19, e passar pela Vila Planalto, 21. A Tabela 3 compara as quatro rotas mais baratas e a Figura 2 destaca a ótima.

Tabela 3 – Rotas mais baratas entre Lago Norte e Lago Sul

Rota	Custo
Lago Norte $\rightarrow^{8}$ Esplanada $\rightarrow^{5}$ Asa Sul $\rightarrow^{5}$ Lago Sul	**8 + 5 + 5 = 18**
Lago Norte $\rightarrow^{8}$ Esplanada $\rightarrow^{11}$ Lago Sul	8 + 11 = 19
Lago Norte $\rightarrow^{7}$ Vila Planalto $\rightarrow^{14}$ Lago Sul	7 + 14 = 21
Lago Norte $\rightarrow^{7}$ Vila Planalto $\rightarrow^{10}$ Esplanada $\rightarrow^{5}$ Asa Sul $\rightarrow^{5}$ Lago Sul	7 + 10 + 5 + 5 = 27

Fonte: elaborado pelo autor.

Figura 2 – Menor caminho entre Lago Norte e Lago Sul (custo 18)



Fonte: elaborado pelo autor (saída do programa em Python).

3.3 Custo dos caminhos entre a sede e os pontos de distribuição

O enunciado pede o custo para percorrer todos os caminhos entre a sede, no Lago Norte, e cada ponto de distribuição. O programa responde em três níveis complementares: o menor custo até cada ponto, a enumeração de todos os caminhos possíveis e a Árvore Geradora Mínima.

3.3.1 Menor custo da sede até cada ponto

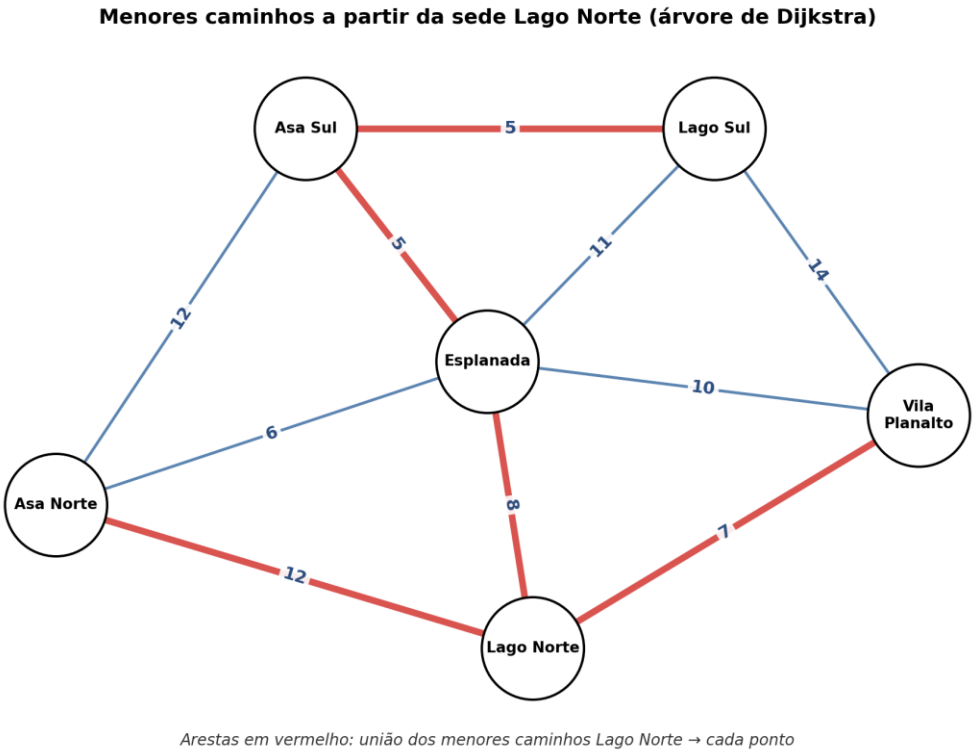
Tabela 4 – Menor custo do Lago Norte até cada ponto (Dijkstra)

Destino	Menor caminho	Custo
Vila Planalto	Lago Norte $\rightarrow^{7}$ Vila Planalto	7
Esplanada	Lago Norte $\rightarrow^{8}$ Esplanada	8
Asa Norte	Lago Norte $\rightarrow^{12}$ Asa Norte	12
Asa Sul	Lago Norte $\rightarrow^{8}$ Esplanada $\rightarrow^{5}$ Asa Sul	13
Lago Sul	Lago Norte $\rightarrow^{8}$ Esplanada $\rightarrow^{5}$ Asa Sul $\rightarrow^{5}$ Lago Sul	18
Total	**Soma dos menores custos a partir da sede**	**58**

Fonte: elaborado pelo autor.

Observa-se que para a Asa Norte a ligação direta (12) é mais vantajosa do que passar pela Esplanada (8 + 6 = 14), e que para chegar à Asa Sul e ao Lago Sul compensa passar pela Esplanada. A Figura 3 mostra a árvore formada pela união desses menores caminhos.

Figura 3 – Árvore de menores caminhos a partir da sede no Lago Norte



Fonte: elaborado pelo autor (saída do programa em Python).

3.3.2 Todos os caminhos possíveis

Com `nx.all_simple_paths`, que faz uma busca em profundidade, o programa listou todos os caminhos simples (sem repetir ponto) entre a sede e cada destino, totalizando **61 caminhos**. A listagem completa, com o custo de cada caminho, está no console; a Tabela 5 resume os valores por destino.

Tabela 5 – Caminhos simples entre o Lago Norte e cada ponto

Destino	Nº de caminhos	Menor custo	Maior custo	Soma dos custos
Asa Sul	14	13	50	431
Lago Sul	14	18	53	437
Esplanada	9	8	53	272
Asa Norte	12	12	49	400
Vila Planalto	12	7	54	434
Total	**61**	–	–	**1974**

Fonte: elaborado pelo autor.

Trecho da saída do console (destino Lago Sul):

```
-> Lago Sul: 14 caminhos
   01. custo 18 | Lago Norte -> Esplanada -> Asa Sul -> Lago Sul
   02. custo 19 | Lago Norte -> Esplanada -> Lago Sul
   03. custo 21 | Lago Norte -> Vila Planalto -> Lago Sul
   04. custo 27 | Lago Norte -> Vila Planalto -> Esplanada -> Asa Sul -> Lago Sul
   05. custo 28 | Lago Norte -> Asa Norte -> Esplanada -> Asa Sul -> Lago Sul
   ...
```

3.3.3 Árvore Geradora Mínima

Se a pergunta for qual o menor conjunto de vias que mantém a sede ligada a todos os pontos, a resposta é a Árvore Geradora Mínima calculada pelo algoritmo de Prim: Asa Sul–Esplanada (5), Asa Sul–Lago Sul (5), Esplanada–Lago Norte (8), Esplanada–Asa Norte (6), Vila Planalto–Lago Norte (7), com custo total **31**. Neste grafo ela coincide com a árvore de menores caminhos da Figura 3, exceto pela Asa Norte, que a AGM liga pela Esplanada (6) em vez de pela ligação direta com a sede (12).

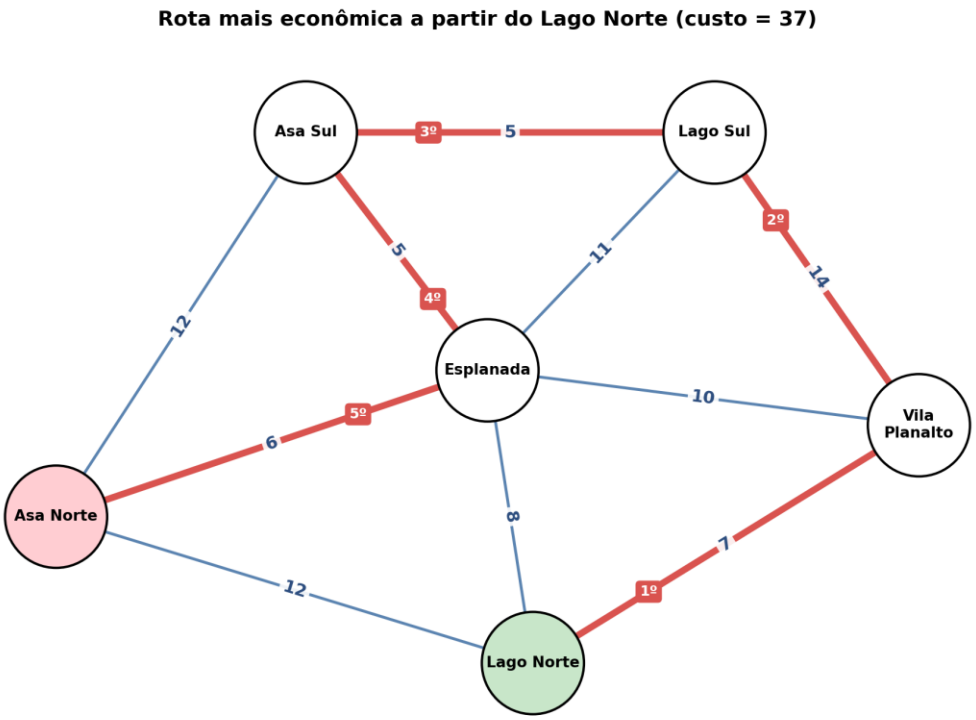
3.4 Rota mais econômica partindo do Lago Norte

Para que Carlos saia da sede e passe por todos os pontos, o programa avaliou as 120 ordens de visita possíveis. A rota de menor custo é:

Lago Norte $\rightarrow^{7}$ Vila Planalto $\rightarrow^{14}$ Lago Sul $\rightarrow^{5}$ Asa Sul $\rightarrow^{5}$ Esplanada $\rightarrow^{6}$ Asa Norte

com custo total **7 + 14 + 5 + 5 + 6 = 37**. Permitir que o veículo repasse por um ponto já visitado não reduziu esse valor (o melhor passeio também custa 37). A estratégia é seguir primeiro para a Vila Planalto, cobrir o lado sul e terminar na Asa Norte, deixando para o fim os trechos baratos (5, 5 e 6) em torno da Esplanada. Caso Carlos precise voltar à sede ao final, a ligação Asa Norte–Lago Norte (12) fecha o ciclo com custo 49.

Figura 4 – Rota mais econômica a partir do Lago Norte (custo 37)



Fonte: elaborado pelo autor (saída do programa em Python).

3.5 Custo em reais: da Esplanada, por todos os pontos, até o Lago Norte

Neste cenário o veículo sai da Esplanada, visita Asa Norte, Asa Sul, Lago Sul e Vila Planalto e termina na sede, no Lago Norte, considerando que uma unidade de custo vale R$ 20,00. O programa encontrou duas respostas, conforme a regra adotada:

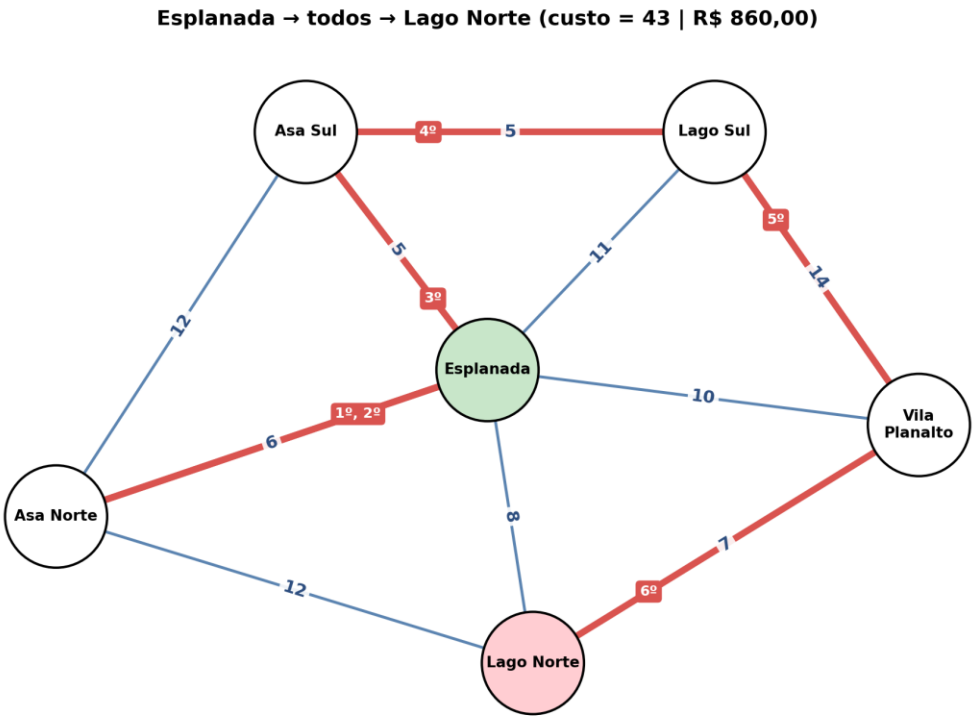
Tabela 6 – Rota da Esplanada ao Lago Norte passando por todos os pontos

Cenário	Rota	Custo	Valor
Sem repetir pontos (caminho hamiltoniano)	Esplanada $\rightarrow^{6}$ Asa Norte $\rightarrow^{12}$ Asa Sul $\rightarrow^{5}$ Lago Sul $\rightarrow^{14}$ Vila Planalto $\rightarrow^{7}$ Lago Norte	6 + 12 + 5 + 14 + 7 = 44	R$ 880,00
Repassando pela Esplanada (passeio ótimo)	Esplanada $\rightarrow^{6}$ Asa Norte $\rightarrow^{6}$ Esplanada $\rightarrow^{5}$ Asa Sul $\rightarrow^{5}$ Lago Sul $\rightarrow^{14}$ Vila Planalto $\rightarrow^{7}$ Lago Norte	6 + 6 + 5 + 5 + 14 + 7 = 43	**R$ 860,00**

Fonte: elaborado pelo autor.

A diferença está no trecho entre a Asa Norte e a Asa Sul: a ligação direta custa 12, enquanto voltar pela Esplanada custa 6 + 5 = 11. Assim, **Carlos gastará R$ 860,00** (43 × R$ 20,00) na rota ótima, economizando R$ 20,00 por viagem em relação à melhor rota que não repete pontos (R$ 880,00). A Figura 5 mostra a rota ótima; o trecho Esplanada–Asa Norte é percorrido duas vezes (1º e 2º).

Figura 5 – Rota da Esplanada ao Lago Norte por todos os pontos (R$ 860,00)



Fonte: elaborado pelo autor (saída do programa em Python).

Trecho da saída do console para o item:

```
Cenário 1 - Caminho Hamiltoniano (sem repetir pontos):
  Esplanada -(6)-> Asa Norte -(12)-> Asa Sul -(5)-> Lago Sul -(14)-> Vila Planalto
    -(7)-> Lago Norte
  Custo 44 x R$ 20,00 = R$ 880,00
Cenário 2 - Passeio ótimo (pode repassar pela Esplanada):
  Esplanada -(6)-> Asa Norte -(6)-> Esplanada -(5)-> Asa Sul -(5)-> Lago Sul -(14)->
    Vila Planalto -(7)-> Lago Norte
  Custo 43 x R$ 20,00 = R$ 860,00
Economia do cenário 2: R$ 20,00
```

4 CONCLUSÃO

A atividade mostrou, na prática, como um problema real de custo logístico pode ser traduzido em um grafo ponderado e resolvido com algoritmos clássicos. O software respondeu a todas as questões propostas: o menor caminho entre Lago Norte e Lago Sul custa 18; os menores custos da sede a cada ponto somam 58, em um universo de 61 caminhos simples possíveis; a rota mais econômica que parte da sede e visita todos os pontos custa 37; e a viagem da Esplanada, por todos os pontos, até o Lago Norte custa R$ 860,00.

As principais dificuldades encontradas no desenvolvimento foram:

- **Interpretar o enunciado.** Alguns itens admitem mais de uma leitura. Em "percorrer todos os caminhos entre o Lago Norte e os pontos", não fica claro se o pedido é o menor custo até cada ponto, a lista de todas as rotas ou o custo da rede; optou-se por apresentar as três visões. No item 3.5, foi preciso decidir se o veículo pode repassar por um ponto já visitado, o que levou a calcular os dois cenários e mostrar a diferença de R$ 20,00.
- **Distinguir problemas fáceis de difíceis.** O menor caminho entre dois pontos é resolvido de forma eficiente por Dijkstra, mas visitar todos os pontos é uma variação do Caixeiro Viajante, NP-difícil. Entender que a força bruta só é viável porque a rede é pequena (120 permutações) — e que não escalaria para centenas de pontos — exigiu estudo além do uso das bibliotecas.

REFERÊNCIAS

CORMEN, Thomas H. *et al*. **Algoritmos**: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

GOLDBARG, Marco; GOLDBARG, Elizabeth. **Grafos**: conceitos, algoritmos e aplicações. Rio de Janeiro: Elsevier, 2012.

HAGBERG, Aric A.; SCHULT, Daniel A.; SWART, Pieter J. Exploring network structure, dynamics, and function using NetworkX. *In*: PYTHON IN SCIENCE CONFERENCE, 7., 2008, Pasadena. **Proceedings** [...]. Pasadena: SciPy, 2008. p. 11-15.

HUNTER, John D. Matplotlib: a 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90-95, 2007.

PYTHON SOFTWARE FOUNDATION. **itertools**: functions creating iterators for efficient looping. Disponível em: https://docs.python.org/3/library/itertools.html. Acesso em: 25 set. 2026.